



DATASET ACQUISITION, BUILD, QA, PACKAGING, AND DELIVERY

Dataset Operations Platform Blueprint

A technically detailed operating and software architecture plan for Caudals to manage proprietary data supply, build ML-ready datasets, and deliver trustworthy data products for early B2B clients.

- Private admin first
- Rights and provenance by design
- Multimodal dataset build system
- Marketplace-ready, not marketplace-exposed

COMPANY

Caudals

DATE

2026-05-08

SCOPE

Internal operations platform

AUDIENCE

Founders, operators, engineers

How To Read This Blueprint

This document translates the Caudals B2B pivot into a concrete platform design. It assumes the near-term product is a high-touch managed dataset operations service with a public landing page, contact form, blog, and private admin dashboard. It does not recommend exposing public marketplace browse, buyer self-service, supplier portals, checkout, or individual sample uploads in the current phase.

Primary Answer
Build Caudals as a Dataset Operations OS: a private control plane that turns buyer demand and supplier data assets into versioned, documented, legally cleared, QA-scored, ML-ready dataset releases.

Key Product Rule
Treat every dataset as a product release, not a file export. A release is only complete when data artifacts, rights evidence, privacy assessment, lineage, schema, quality score, documentation, pricing, and delivery evidence exist together.

Table Of Contents

01	Executive Blueprint	02	Platform Principles
03	Reference Architecture	04	Technology Stack
05	Domain Model	06	Dataset Operations Pipeline
07	Modality Playbooks	08	Dashboards And Internal Tools
09	Security, Privacy, And Trust	10	Implementation Roadmap
11	Metrics And Operating Model	12	Appendices And Sources

Legal note
This is an engineering and product blueprint, not legal advice. Rights, consent, data protection, and AI-training use permissions should be reviewed by qualified counsel before publication or buyer delivery.

01

Executive Blueprint

Caudals should prioritize the internal system that makes high-trust dataset delivery repeatable. Marketplace exposure can wait; the operational control plane is the current product.

What Buyers Need

Custom or hard-to-access data that is usable for model training, fine-tuning, evaluation, benchmarking, enrichment, or RAG.

- Clear permitted use and AI-training rights.
- Fast feasibility decisions and samples.
- ML-ready packaging and documentation.
- Quality, limitations, and lineage evidence.

What Suppliers Need

A managed path to monetize proprietary data without building sales, privacy, packaging, delivery, and buyer-support operations.

- Safe rights and sensitivity review.
- Low-friction data-room intake.
- Revenue share and listing readiness.
- Controls over what is exposed or sold.

What Caudals Needs

An operator-grade platform for sourcing, rights review, ingestion, profiling, cleaning, anonymization, labeling, QA, packaging, and delivery.

- Auditable state transitions.
- Reusable build pipelines.
- Dataset-level unit economics.
- Visible blockers and acceptance gates.

Strategic Architecture Decision

DECISION	RECOMMENDATION	RATIONALE
Product surface	Private admin dashboard first.	Early revenue depends on managed delivery quality, not self-service marketplace breadth.
Data plane	Object storage with staged zones: raw, profiled, cleaned, labeled, packaged, delivered.	Supports large files, multimodal assets, immutable raw preservation, and buyer-specific package outputs.
Control plane	Supabase Postgres metadata, RLS, audit log, and server actions/API routes.	Fits current stack while enforcing scoped access and operational traceability.
Workflow engine	Start with job tables and workers, then move core builds into Dagster asset orchestration.	Dagster's asset model fits dataset products and makes lineage, observability, and re-materialization first-class.
Dataset standards	Ship Dataset Cards, Croissant JSON-LD manifests, SPDX license identifiers, and W3C PROV-inspired lineage.	Improves buyer trust, ML tool compatibility, search/catalog readiness, and reproducibility.
Quality gates	No release without rights, privacy, schema, quality, label, packaging, and delivery approvals.	Prevents the platform's highest-risk failure modes: illegal listing, PII leakage, low-quality delivery, and unclear buyer use rights.

Near-Term Product

- Lead triage for buyer demand and supplier monetization interest.
- Structured buyer briefs and supplier asset records.
- Dataset build board with stages, owners, blockers, and gates.
- Pipeline run registry for ingestion, profiling, cleaning, PII, labeling, QA, and packaging.
- Release package generator with manifests, quality report, dataset card, and delivery evidence.

Explicit Non-Goals

- No individual sample-upload workflow.
- No public buyer workspace until requirements, pricing, and fulfillment are redesigned.
- No supplier portal until rights capture and source access flows are safe.
- No public marketplace catalog until every listing has rights, privacy, provenance, sample, QA, and refresh evidence.
- No payment relaunch until offer, license, delivery, and supplier-share records are reliable.

02

Platform Principles

The platform should be designed around trust, repeatability, and operational throughput. The first clients will judge Caudals by whether datasets arrive usable, documented, licensed, and on time.

Dataset Product Contract

Every sellable dataset version must have:

- Commercial purpose and buyer use case.
- Supplier/source provenance and permitted-use evidence.
- Privacy classification, PII handling decision, and residual risk.
- Data dictionary, schema, modality profile, and format profile.
- Quality scorecard with method, sample size, checks, failures, and known limits.
- Release manifest with checksums, row/object counts, splits, packaging formats, and artifact paths.
- Dataset card and buyer-facing delivery note.

Operational Invariants

- Raw supplier data is immutable after ingestion; later transformations create new artifacts.
- Every stage emits metadata, not just files.
- Every human decision affecting rights, PII, pricing, publication, or delivery is audit logged.
- Pipeline runs are idempotent where possible and restartable where not.
- Buyer-visible materials never imply permissions that are not explicitly cleared.
- Internal admin remains the control room; public marketplace surfaces stay hidden until redesigned.

Required Stage Gates

GATE	REQUIRED BEFORE	EVIDENCE	OWNER
Rights Gate	Any external sample, listing, or delivery.	Source owner, contract, permitted AI use, redistribution limits, prohibited uses.	Operator + legal reviewer
Privacy Gate	Cleaning, labeling vendor export, buyer sample, or buyer release.	PII scan, sensitivity class, minimization decision, anonymization/pseudonymization method, residual risk.	Data engineer + privacy reviewer
Quality Gate	Package release approval.	Expectations, profile deltas, duplicate rate, missingness, label quality, split leakage checks.	Data science lead
Documentation Gate	Buyer handoff or catalog-ready status.	Dataset card, data dictionary, release manifest, limitations, usage notes.	Dataset operator
Delivery Gate	Signed download, S3 share, API, warehouse share, or private transfer.	License, invoice/quote state if applicable, checksum, delivery log, acceptance criteria.	Commercial ops + operator

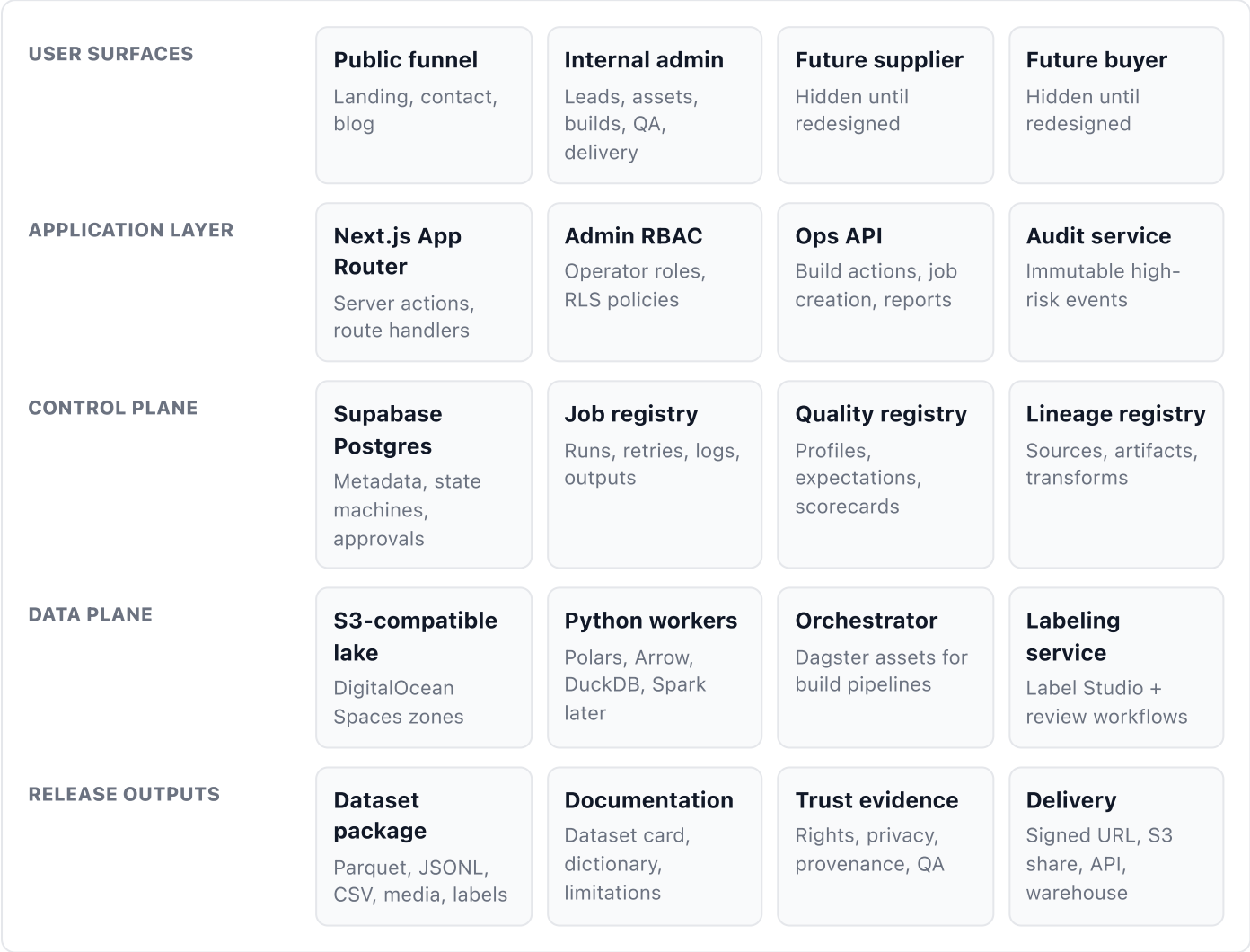
Blueprint posture

The platform should make the unsafe path hard. Operators can move quickly, but the UI and backend should refuse publication or buyer delivery when required rights, PII, QA, documentation, or license records are missing.

03

Reference Architecture

Caudals should separate metadata control from heavy data processing. The Next.js/Supabase app is the operational control plane; object storage and worker services are the data plane.



Service Boundaries

COMPONENT	OWNS	DOES NOT OWN	IMPLEMENTATION NOTES
Next.js admin app	Human workflows, metadata CRUD, approvals, reports, job launch, delivery records.	Large file processing, long-running transformations, heavy profiling.	Keep it TypeScript-first, RLS-aware, and route-restricted under landing mode.
Postgres control plane	Canonical state, users, organizations, assets, build runs, lineage summaries, audit logs.	Large raw data, high-volume media, full transformation outputs.	Use strict RLS and service-role wrappers; keep metadata queryable.
Object data lake	Raw files, transformed artifacts, manifests, logs, previews, release packages.	Authorization decisions, business state, approvals.	Use path allowlists, checksums, retention classes, and short-lived signed URLs.
Dataset worker service	Ingestion, profiling, cleaning, PII scans, dedup, enrichment, labeling sync, packaging.	Direct UI, direct customer access.	Containerized Python service with idempotent jobs and structured result metadata.
Orchestrator	Pipeline graph, scheduling, dependencies, retries, materializations, lineage events.	Product-facing state without synchronization.	Start simple; adopt Dagster for repeatable build assets once MVP job tables are stable.

Storage Zones

```
s3://caudals-data/{env}/
  suppliers/{supplier_id}/assets/{asset_id}/raw/{ingestion_id}/
  builds/{build_id}/bronze/{artifact_id}/
  builds/{build_id}/silver/{artifact_id}/
  builds/{build_id}/gold/{dataset_version_id}/
  builds/{build_id}/labeling/{label_batch_id}/
  releases/{dataset_version_id}/{package_release_id}/
  deliveries/{delivery_id}/
  quarantine/{risk_case_id}/
```

Raw supplier data is written once and never edited in place. Every derivative artifact carries a content hash, code version, input artifact IDs, and transformation run ID. Buyer-specific packages are separate from canonical gold dataset versions so commercial terms and permitted fields can differ by buyer.

Technology Stack

Keep the existing web stack, add a Python data operations plane, and delay heavyweight lakehouse complexity until data volume, concurrency, and client requirements justify it.

Recommended Stack By Layer

LAYER	MVP CHOICE	SCALE CHOICE	WHY
Web/admin	Existing Next.js 16, React 19, TypeScript, Tailwind v4, Radix/shadcn primitives.	Same, with deeper RBAC, audit components, and admin workflow primitives.	Matches repo architecture and avoids rebuilding foundations.
Metadata	Supabase Postgres with RLS, migrations, audit tables.	Read replicas, partitioned audit/event tables, optional warehouse mirror.	Postgres is the right control plane for operational state and trust evidence.
Object storage	DigitalOcean Spaces with private buckets, signed URLs, path validation, checksums.	S3-compatible multi-bucket topology, lifecycle tiers, cross-region backup.	Large datasets belong in object storage, not Postgres.
Data processing	Python 3.12 workers using Polars, PyArrow, DuckDB, Pandas where needed.	Spark/Ray only for workloads that exceed single-node or DuckDB/Polars limits.	Most pilots can be fast and cheap on columnar local compute before distributed processing.
File/table formats	Parquet for tabular, JSONL for LLM/text, media objects plus metadata indexes.	Delta Lake or Apache Iceberg for large mutable analytic tables after storage semantics are tested.	Parquet/Arrow are universal; open table formats add ACID/time travel at higher operational cost.
Orchestration	Postgres job registry plus worker queues for first implementation.	Dagster assets with OpenLineage events and external worker execution.	Start low-friction, then move repeatable dataset builds into an asset graph.
Data quality	Custom scorecards plus Great Expectations/Pandera validation suites.	Quality registry with trend dashboards, anomaly detection, and release blocking.	Validation needs to become a release gate, not an ad hoc notebook.
PII/privacy	Microsoft Presidio for text PII detection/anonymization, plus modality-specific scanners.	Policy-driven DLP with reviewer queues, residual-risk scoring, and re-identification testing.	PII detection must be built into ingestion and release gates.
Labeling	Label Studio for task configuration, labeling, review, and annotation export.	Integrated label ops with active learning, consensus scoring, and vendor management.	Supports multiple modalities and avoids building a labeling UI from scratch.
Metadata standards	Dataset Card, Croissant JSON-LD, SPDX license identifiers, release manifest.	DCAT catalog mappings and W3C PROV/OpenLineage export.	Improves portability, buyer review, catalog readiness, and auditability.

Recommended Repository Shape

```
app/
  (app)/admin/dataset-ops/...      # private internal modules
lib/actions/dataset-ops/...      # server actions and service wrappers
lib/dataset-ops/...              # TypeScript domain validation, state machines
supabase/migrations/...         # new B2B metadata model
services/
  dataset-worker/                # Python worker service
  caudals_ops/
    jobs/                        # ingestion, profiling, pii, labeling, packaging
    quality/
    manifests/
    connectors/
    privacy/
  tests/
docs/blueprints/                # report artifacts and platform blueprints
```

Why Not Put Data Processing In Next.js?

The web app should stay responsive and secure. Large file transforms, PII scans, labeling export imports, and package generation need isolated workers with explicit resource limits, resumable execution, structured logs, and no browser-exposed secrets.

Why Not Build A Marketplace First?

A marketplace without operational guarantees creates risk: unclear rights, inconsistent quality, stale listings, and buyer disappointment. The marketplace should be a projection of verified dataset products, not the source of truth.

Domain Model

The current schema is legacy from the pre-pivot product. The B2B operations platform needs a new metadata model around organizations, supplier assets, buyer briefs, dataset builds, artifacts, rights, privacy, QA, releases, and delivery.

Core Entities

ENTITY	PURPOSE	CRITICAL FIELDS
organizations	Buyer, supplier, partner, vendor, internal organization records.	type, legal_name, country, industry, risk_tier, status.
contacts	People attached to organizations and workflows.	role, email, authority_level, communication_preferences.
buyer_briefs	Structured buyer dataset needs.	use_case, modality, domain, geography, freshness, volume, budget, deadline, sensitivity_constraints.
supplier_assets	Potential or approved supplier-owned data assets.	supplier_id, asset_type, access_method, rights_status, pii_status, freshness, update_cadence, commercial_fit_score.
data_sources	Specific source systems, files, APIs, shares, or exports within an asset.	source_kind, location_ref, credential_ref, schema_hint, source_owner, ingestion_policy.
rights_reviews	Rights, consent, license, redistribution, and AI-use review.	permitted_ai_use, redistribution_allowed, territory, field_restrictions, reviewer_id, expires_at.
privacy_assessments	PII, sensitive category, minimization, anonymization, and residual-risk state.	pii_classes, sensitivity_tier, method, residual_risk, approved_for_labeling, approved_for_delivery.
dataset_projects	Commercial and operational project container.	tier, buyer_brief_id, primary_supplier_id, owner_id, status, target_release_date.
dataset_builds	Build attempt or version track under a project.	stage, stage_status, blocked_reason, target_schema_id, acceptance_criteria.
data_artifacts	Raw and derived files, tables, media sets, reports, and packages.	artifact_kind, zone, storage_uri, checksum, record_count, byte_size, parent_artifact_ids.
pipeline_runs	Job execution state for ingestion, profiling, cleaning, PII, labeling, packaging.	job_type, status, input_artifacts, output_artifacts, code_version, started_at, finished_at, error_summary.
quality_reports	Profile, validation, score, and release-blocking quality evidence.	score, dimensions, checks_passed, checks_failed, sample_method, known_limitations.

ENTITY	PURPOSE	CRITICAL FIELDS
labeling_projects	Ontology, labeling task, annotator, review, and consensus state.	ontology_version, tool_project_ref, task_count, consensus_method, review_status.
dataset_versions	Canonical build output before buyer-specific packaging.	semantic_version, content_hash, schema_version, split_strategy, release_candidate_status.
package_releases	Deliverable dataset package with documentation and manifest.	format_profile, manifest_uri, dataset_card_uri, license_summary_uri, approval_status.
deliveries	Buyer delivery events and acceptance evidence.	buyer_id, delivery_method, access_expires_at, checksum_verified, accepted_at, revoked_at.
audit_events	Immutable high-risk activity record.	actor_id, entity_type, entity_id, event_type, before, after, reason, ip_hash.

State Machines

Dataset Build Stages

scoped -> rights_pending -> ingesting
-> profiling -> cleaning -> pii_review
-> labeling -> qa -> packaging ->
delivery_ready -> delivered ->
accepted

Release Approval States

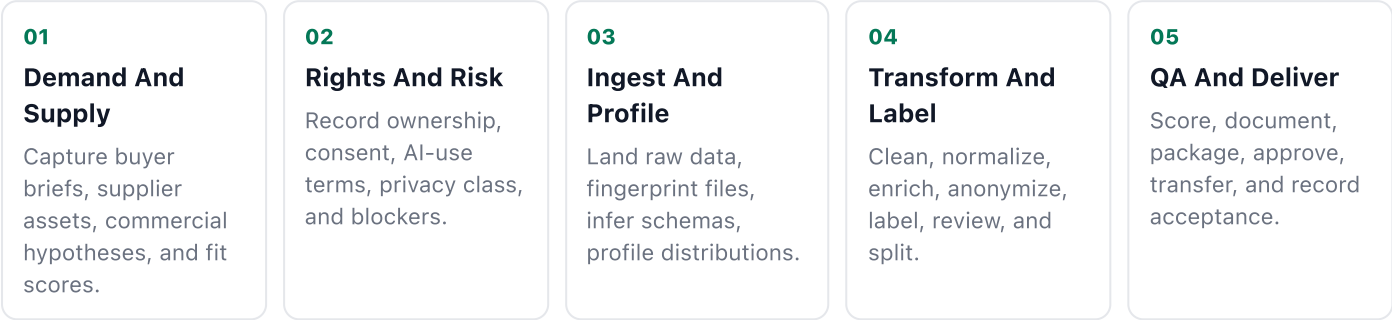
draft -> qa_pending -> rights_pending
-> privacy_pending -> approved ->
delivered -> deprecated

Relational Design Rules

- Use explicit status enums or constrained text values; do not infer status from missing fields.
- Every high-risk approval needs approved_by, approved_at, approval_basis, and evidence_uri.
- Every storage object referenced in metadata needs a checksum, byte size, content type, and retention policy.
- Every worker-produced artifact links to the pipeline run, code version, input artifacts, and environment.
- Use append-only audit events for rights, privacy, publication, pricing, delivery, and access changes.

Dataset Operations Pipeline

The pipeline should be modality-agnostic at the control-plane level and modality-specific in worker plugins. A tabular data build, image detection build, text corpus build, and multimodal build should all move through the same gates while using different processors.



Detailed Workflow

STAGE	OPERATOR ACTIONS	SYSTEM OUTPUTS	RELEASE GATE
1. Qualification	Score buyer need or supplier asset by use case, urgency, data scarcity, rights feasibility, budget, and build complexity.	Structured brief or supplier asset, initial risk tier, project proposal, next action.	Qualified owner and commercial hypothesis.
2. Feasibility	Assess source availability, permitted uses, sample quality, volume, modality, processing cost, legal blockers, and delivery options.	Feasibility memo, risk register, target schema, acceptance criteria, estimated timeline and cost.	Go, pilot, pause, or reject decision.
3. Secure Intake	Create data-room path, collect files/API credentials/share access, verify checksums, quarantine suspicious files.	Ingestion run, raw artifacts, checksums, storage paths, source manifest.	Raw data locked and source owner confirmed.
4. Profiling	Run schema inference, type detection, missingness, duplicates, distributions, media metadata extraction, language detection, and volume checks.	Schema profile, data profile, anomaly notes, recommended cleaning plan.	Profile reviewed and target transformations approved.
5. Rights And PII Review	Verify contracts, consent basis, field-level restrictions, PII classes, minimization opportunities, and labeling/export constraints.	Rights review, privacy assessment, field policy map, redaction plan.	Rights and privacy gates clear for the next stage.
6. Cleaning And Normalization	Fix encoding, normalize units, standardize categories, deduplicate, remove corrupt items, align timestamps, repair malformed records.	Silver artifacts, cleaning run logs, data diffs, rejected-record table.	Validation checks pass or exceptions are approved.
7. Enrichment	Add public signals, derived features, geocodes, taxonomy mappings, embeddings, weak labels, or entity normalization where useful.	Enriched artifacts, enrichment provenance, source citations, confidence fields.	Enrichment rights and quality reviewed.
8. Ontology And Labeling	Define labels/classes, write labeling guidelines, create tasks, import pre-labels, assign annotators, review consensus and edge cases.	Ontology version, label batches, annotation exports, reviewer decisions, label quality report.	Label quality threshold met and guideline version frozen.
9. Split And Leakage Review	Define train/validation/test split by entity/time/geography/source as appropriate; block duplicate or related entities across splits.	Split manifest, leakage checks, class balance report, reproducible random seed.	Split strategy approved for buyer use case.

STAGE	OPERATOR ACTIONS	SYSTEM OUTPUTS	RELEASE GATE
10. Packaging	Generate buyer-specific package, manifests, docs, previews, checksums, examples, and delivery notes.	Release package, dataset card, Croissant JSON-LD, data dictionary, QA scorecard.	Release approved by rights, privacy, QA, and commercial owners.
11. Delivery And Acceptance	Provision signed URL/S3 share/API/warehouse delivery, verify checksum, track buyer download and acceptance.	Delivery event, access log, acceptance evidence, support ticket link if needed.	Accepted, remediated, or rejected with evidence.

Dataset Quality Score

Use a 100-point internal score that is visible in admin but buyer-facing only after review. Weighting should be configurable by modality and buyer use case.

Default Weighting		Hard Release Blocks	
Completeness	15%	- Unknown or uncleared AI-training rights. - Unknown PII status or unapproved sensitive fields. - No release manifest or checksum coverage. - No data dictionary for tabular/text fields or no label ontology for supervised datasets. - Critical Great Expectations/Pandera checks failing without documented exception. - Unreviewed label batches below consensus threshold. - Buyer-specific restrictions not reflected in package contents.	
Validity and schema conformance	15%		
Uniqueness and deduplication	10%		
Representativeness and balance	15%		
Freshness and temporal coverage	10%		
Label accuracy and reviewer agreement	15%		
Privacy and rights readiness	10%		
Documentation and reproducibility	10%		

Modality Playbooks

The control plane should stay consistent while processing plugins vary by data type. The platform must support tabular, text, image, video, audio, geospatial, time-series, and multimodal builds.

MODALITY	COMMON INPUTS	PROCESSING FOCUS	DELIVERY FORMATS
Tabular / transactional	CSV, Excel, database exports, warehouse shares, APIs.	Schema inference, unit normalization, type repair, duplicate detection, joins, missingness, outliers, temporal leakage.	Parquet, CSV, Arrow, DuckDB, SQL dump, warehouse share, JSONL for row-level ML tasks.
Text / documents	Support tickets, PDFs, emails, knowledge bases, contracts, articles.	OCR, encoding, language detection, document chunking, dedup, PII redaction, taxonomy labels, prompt/eval pairs.	JSONL, Parquet, Markdown, TXT, prompt-completion JSONL, RAG corpus manifest.
Images	Inspection photos, product images, satellite tiles, medical or industrial imagery.	EXIF stripping/review, perceptual dedup, resolution normalization, bounding boxes, segmentation masks, class balance.	COCO, YOLO, Pascal VOC, WebDataset shards, Parquet metadata plus object paths.
Video	Dashcam, inspection, security, sports, manufacturing, field operations.	Scene/clip extraction, frame sampling, temporal labels, blur/quality scoring, privacy masking, large-object packaging.	MP4 clips plus JSONL/Parquet annotations, WebDataset, COCO-style frame annotations.
Audio / speech	Calls, field recordings, voice notes, acoustic sensors.	Transcription, diarization, language/accent metadata, PII redaction in transcripts, audio quality, speaker split leakage.	WAV/FLAC/MP3 plus JSONL transcripts, timestamps, labels, speaker metadata.
Time-series / IoT	Sensor logs, telemetry, energy, industrial machines, GPS traces.	Sampling rate, missing intervals, clock drift, unit normalization, anomaly labels, windowing, train/test split by time/entity.	Parquet, Arrow, Timescale/Postgres export, HDF5 where required, JSONL windows.
Geospatial	Coordinates, shapes, tiles, routes, parcels, environmental layers.	CRS normalization, geocoding, spatial joins, privacy aggregation, tile generation, temporal coverage.	GeoParquet, GeoJSON, Shapefile where required, COG for raster, vector tiles.
Multimodal	Text + images, product records + images, audio + transcripts, video + sensor events.	Stable entity IDs, cross-modal alignment, synchronized splits, modality-specific PII, missing modality handling.	Manifest-first package with per-modality artifacts, JSONL examples, Parquet metadata, media object paths.

Shared Manifest Pattern

```
{
  "dataset_id": "ds_...",
  "dataset_version": "1.0.0",
  "release_id": "rel_...",
  "modality": ["tabular", "image"],
  "license_summary": "Private commercial license; AI training allowed for named buyer use case.",
  "rights_review_id": "rr_...",
  "privacy_assessment_id": "pa_...",
  "quality_report_id": "qr_...",
  "artifacts": [
    {
      "role": "train_metadata",
      "format": "parquet",
      "uri": "s3://.../train.parquet",
      "sha256": "...",
      "records": 48000
    },
    {
      "role": "images",
      "format": "webdataset",
      "uri": "s3://.../images-000.tar",
      "sha256": "...",
      "objects": 12000
    }
  ],
  "splits": {"train": 0.8, "validation": 0.1, "test": 0.1},
  "created_at": "2026-05-08T00:00:00Z"
}
```

Dashboards And Internal Tools

The admin dashboard should become the operational control room. It should be dense, calm, and trust-oriented: every screen should expose owner, next action, stage, blockers, rights, PII, QA, and delivery state.

Required Admin Modules

MODULE	PRIMARY USERS	KEY VIEWS	CRITICAL ACTIONS
Command Center	Founders, operators	Active projects, stage counts, blockers, upcoming due dates, QA risk, delivery readiness.	Assign owner, escalate blocker, open build, approve stage transition.
Leads And Briefs	Sales, operators	Buyer briefs, supplier leads, qualification score, next action, conversion stage.	Create feasibility study, convert to project, reject, request missing info.
Organizations	Sales, operations, legal	Buyer/supplier profiles, contacts, contracts, risk, communication timeline.	Add contact, link contract, set risk tier, attach notes.
Supplier Assets	Data ops, legal, privacy	Available data assets, source systems, rights, PII, freshness, update cadence, commercial fit.	Create source, request sample, run profile, start rights review, approve for pilot.
Dataset Builds	Operators, engineers, data scientists	Stage board, timeline, blockers, artifacts, runs, acceptance criteria, build notes.	Start pipeline job, mark stage review, attach artifact, add blocker, create release candidate.
Pipeline Runs	Engineers, operators	Job status, logs, retries, input/output artifacts, duration, failure summary.	Retry, cancel, inspect artifacts, promote outputs, quarantine outputs.
Quality And Profiling	Data scientists, QA reviewers	Schema profile, quality score, failed checks, distribution deltas, label quality, leakage checks.	Approve exception, require rework, publish scorecard, compare versions.
Label Operations	Label managers, reviewers	Ontology versions, labeling batches, annotator progress, consensus, gold samples, edge cases.	Create batch, export tasks, import annotations, assign review, approve labels.
Rights And Privacy	Legal/privacy reviewers, operators	Pending reviews, field restrictions, PII classes, redaction outcomes, expiration dates.	Approve, block, request counsel, expire review, require minimization.
Releases And Delivery	Commercial ops, operators	Release candidates, package formats, docs, checksums, delivery method, buyer acceptance.	Generate package, approve release, create signed delivery, revoke access, record acceptance.

Dataset Build Detail Screen

Top Rail

- Project name, buyer/supplier, owner, due date, tier, stage.
- Status chips: rights, PII, QA, labeling, package, delivery.
- Primary next action and blocker summary.

Tabs

- Overview, Requirements, Sources, Pipeline, Artifacts, QA, Labels, Rights, Privacy, Release, Activity.
- Each tab should have empty, loading, error, and success states.
- Danger actions require reason capture and audit event.

Design Guidance

- Light mode, white panels, subtle borders, black typography, gray structure, teal status accents.
- No marketing hero patterns inside admin. Prioritize scan speed, tables, filters, status chips, and compact forms.
- Use stable dimensions for status columns, pipeline step controls, and score widgets to avoid layout shift.
- Every critical state should be visible without opening a modal: rights, provenance, PII, QA, license, package, delivery.

Security, Privacy, And Trust

Caudals is selling trust in addition to data. Security and privacy controls must be visible in product state, enforced by backend gates, and backed by immutable evidence.

Access Model

ROLE	ALLOWED	RESTRICTED
Admin	Manage users, roles, settings, high-risk approvals.	Direct raw data download without purpose and audit record.
Operator	Manage leads, projects, assets, stage state, comments, delivery coordination.	Rights/privacy approvals unless assigned reviewer permission.
Data engineer	Run pipelines, inspect profiles, promote artifacts, view technical logs.	Commercial pricing, contracts, unrelated supplier data.
Data scientist / QA	Review quality, labels, splits, metrics, known limitations.	Raw PII fields unless approved for the project.
Legal/privacy reviewer	Approve or block rights and privacy assessments.	Mutating pipeline artifacts or delivery packages.
Commercial ops	Quotes, contracts, licenses, invoices, delivery acceptance, supplier share.	Changing quality or privacy evidence.

Control Requirements

Data Access

- RLS on all exposed metadata tables.
- Service-role access only through scoped server wrappers.
- Signed URLs with short TTLs and project-scoped authorization checks.
- Object paths validated against known artifact records.
- Raw and PII-bearing data access requires reason capture.

Privacy And PII

- Run PII scans during ingestion and before labeling/export/release.
- Separate anonymization, pseudonymization, redaction, encryption, and minimization decisions.
- Treat pseudonymized data as still sensitive unless counsel determines otherwise.
- Store salts/keys outside the repo and outside dataset packages.
- Maintain field-level delivery policy for every package.

Provenance And Lineage

- Track source, activity, agent, artifact, and derivation relationships.
- Capture pipeline code version, environment, input checksums, output checksums.
- Export OpenLineage-compatible run metadata when orchestration matures.
- Use W3C PROV concepts for buyer-facing lineage summaries.

Release Assurance

- Release packages require rights, privacy, QA, documentation, and commercial approval.
- Every buyer delivery includes checksum verification and license summary.
- Releases can be deprecated, revoked, or superseded with explicit reason.
- Access logs and acceptance evidence remain linked to package version.

Risk Register Template

RISK	EXAMPLE TRIGGER	MITIGATION	RELEASE DECISION
Unclear AI-training rights	Supplier contract permits analytics but not model training.	Block delivery until contract addendum or field removal.	Hard block.
PII or sensitive data leakage	Emails, faces, plates, GPS trails, identifiers remain in package.	Minimize fields, redact/mask, aggregate, anonymize, or reject use case.	Hard block unless approved residual risk exists.
Label noise	Low reviewer agreement or Cleanlab-like issue detection flags many labels.	Rework ontology, add gold set, increase consensus review.	Block supervised training release above threshold.
Data leakage across splits	Same entity, media sequence, or document family appears in train and test.	Group-aware split, time-aware split, duplicate cluster split.	Block model-eval package.
Supplier reputation risk	Buyer-visible data reveals sensitive commercial behavior.	Aggregate, delay, redact, segment package, or use private offer only.	Reviewer decision required.

Implementation Roadmap

The next implementation phases should build the internal platform in thin vertical slices. Each phase must produce usable operator functionality and reduce risk for the first client dataset builds.

PHASE	DURATION	DELIVERABLES	EXIT CRITERIA
Phase 39: B2B control-plane schema	1-2 weeks	New migrations for organizations, contacts, buyer briefs, supplier assets, rights reviews, privacy assessments, projects, builds, artifacts, runs, QA, releases, deliveries, audit events.	Migration tests pass; RLS policy tests cover admin/operator roles; legacy tables are not extended for new behavior.
Phase 40: Admin command center	2 weeks	Private admin IA for leads, briefs, supplier assets, dataset build board, blockers, stage state, and audit activity.	Operators can create and move a project from lead to scoped build with visible rights/PII/QA placeholders.
Phase 41: Secure ingestion and profiling	2-3 weeks	Worker service, object storage layout, ingestion jobs, checksum manifests, schema/data profiling, preview generation, source manifests.	A real pilot sample can be ingested, profiled, and reviewed without manual spreadsheet tracking.
Phase 42: Rights, privacy, and quality gates	2-3 weeks	Rights review workflow, PII scan workflow, field policy map, quality expectations, scorecard, hard release blockers.	No release candidate can be approved without rights, privacy, and quality evidence.
Phase 43: Labeling and curation ops	2-4 weeks	Label Studio integration, ontology records, task export/import, reviewer queues, consensus metrics, label quality report.	Image/text/tabular supervised pilot can complete labeling with review evidence.
Phase 44: Packaging and delivery	2-3 weeks	Package generator, Dataset Card, Croissant JSON-LD, release manifest, data dictionary, signed delivery, buyer acceptance records.	One production-ready package can be delivered with manifest, checksums, docs, license summary, and acceptance evidence.
Phase 45: Catalog/private offer readiness	3-5 weeks	Private listing records, sample previews, pricing fields, refresh cadence, supplier revenue share, listing visibility gates.	Internal team can prepare catalog-ready listings without exposing marketplace browse publicly.
Phase 46: Portal pilots	Later	Supplier intake and buyer request surfaces after internal workflows are proven.	Public/self-serve surfaces only expose workflows that the internal platform can fulfill reliably.

MVP Backlog

Must Build Before First Production Delivery

- New metadata schema and audit log.
- Admin dataset build board.
- Source/artifact storage registry with checksums.
- Ingestion and profiling worker.
- Rights and PII review records with release blockers.
- Quality scorecard and release manifest.
- Delivery event and buyer acceptance evidence.

Can Be Manual Initially

- Legal document review, as long as the result is recorded.
- Some cleaning/enrichment scripts, as long as outputs are registered as pipeline runs.
- Specialist annotation vendor coordination, as long as batches and QA are tracked.
- Commercial quotes, as long as package/license/delivery state is linked.

Engineering Acceptance Bar

- Every phase includes migration tests, RLS/access tests, targeted unit tests, and admin UI checks.
- Dataset-impacting changes include synthetic fixtures covering rights-approved, rights-blocked, PII-clear, and PII-blocked cases.
- Worker jobs must be idempotent or safely retryable; failed jobs must leave recoverable metadata.
- Every admin workflow must have explicit loading, empty, error, success, and blocked states.
- Validation logs must include commands, screenshots for UI phases, and sample artifact metadata.

11

Metrics And Operating Model

The platform should make operational throughput and trust visible. Metrics should answer whether Caudals can deliver better datasets faster, with lower risk, and with repeatable margin.

Company Metrics

CATEGORY	METRICS	WHY IT MATTERS
Funnel	Qualified buyer briefs, qualified supplier assets, reply time, discovery conversion, feasibility close rate.	Shows demand/supply health and whether the public funnel is producing sellable opportunities.
Operations	Cycle time by stage, blocker age, rework rate, on-time delivery, active projects per operator.	Shows whether managed service work is becoming repeatable.
Quality	QA pass rate, critical validation failures, duplicate rate, label agreement, data leakage incidents.	Shows whether packages are trustworthy enough for buyer model work.
Trust	Rights pass rate, PII block rate, unresolved privacy risk, release approvals without exceptions.	Shows if Caudals is de-risking supply and protecting buyers/suppliers.
Commercial	Feasibility-to-pilot conversion, pilot-to-complete conversion, gross margin by build, supplier revenue share, repeat/refreshed revenue.	Shows whether operations create profitable reusable data products.

Operating Cadence

Daily Ops

- Review active blockers and stage transitions.
- Check failed pipeline runs.
- Assign next actions for leads and builds.
- Escalate rights or PII blockers.

Weekly Release Board

- Review release candidates.
- Compare quality score trends.
- Approve or reject exceptions.
- Update delivery forecasts.

Monthly Portfolio Review

- Analyze buyer demand patterns.
- Rank supplier assets by sellability.
- Identify reusable catalog candidates.
- Review margins and rework drivers.

Roles And Responsibilities

ROLE	OWNS	ACCOUNTABLE FOR
Dataset Operator	Project state, next action, supplier/buyer coordination, documentation completeness.	Build does not stall and release package is complete.
Data Engineer	Ingestion, profiling, cleaning, transforms, artifact registry, worker reliability.	Artifacts are reproducible, validated, and traceable.
Data Scientist / ML QA	Quality methodology, split strategy, label QA, model-use suitability.	Dataset is fit for intended ML use and limitations are clear.
Label Ops Lead	Ontology, annotation guidelines, annotator throughput, review, consensus.	Labels meet acceptance criteria and edge cases are resolved.
Legal / Privacy Reviewer	Rights, permitted use, PII, redaction/anonymization decisions, residual risk.	Dataset is not released beyond documented permissions.
Commercial Owner	Buyer scope, quote, contract, license, supplier revenue share, acceptance.	Delivery terms match what was built and what rights allow.

Appendices And Sources

The blueprint is based on the Caudals repository context plus current standards and official documentation for provenance, catalog metadata, AI/privacy risk, object storage, data quality, lineage, labeling, PII handling, and ML dataset documentation.

Immediate Build Checklist

CHECKLIST ITEM	DEFINITION OF DONE
Schema design approved	New B2B tables, RLS policies, status enums, audit events, and indexes are reviewed before migrations land.
Storage paths reserved	Spaces bucket layout, naming conventions, retention, signed URL policy, and checksum requirements are documented.
Worker skeleton created	Containerized Python worker can run one ingestion/profile job locally and register output artifacts.
Admin IA implemented	Operators can create buyer brief, supplier asset, dataset project, build, and stage transitions.
Release gate enforced	Backend refuses package approval without rights, privacy, QA, docs, and artifact checksums.
First pilot dry-run	A synthetic multimodal sample moves from ingestion to package release with all evidence generated.

Recommended Source References

1. Caudals repository: [OVERVIEW.md](#) , [AGENTS.md](#) , [docs/ARCHITECTURE.md](#) , [docs/DESIGN.md](#) , [docs/FRONTEND.md](#) , [docs/SECURITY.md](#) , [docs/product-specs/*](#) .
2. W3C PROV Overview: <https://www.w3.org/TR/prov-overview/>
3. W3C Data Catalog Vocabulary DCAT 3: <https://www.w3.org/TR/vocab-dcat-3/>
4. NIST AI Risk Management Framework: <https://www.nist.gov/itl/ai-risk-management-framework>
5. NIST Privacy Framework: <https://www.nist.gov/privacy-framework>
6. GDPR official text, EUR-Lex: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679>
7. ISO/IEC 42001 AI management systems overview: <https://www.iso.org/standard/42001>
8. Apache Parquet overview: <https://parquet.apache.org/docs/overview/>
9. Apache Arrow overview: <https://arrow.apache.org/overview/>
10. Delta Lake documentation: <https://docs.delta.io/>
11. Apache Iceberg specification: <https://iceberg.apache.org/spec/>
12. Dagster documentation: <https://docs.dagster.io/>
13. OpenLineage documentation: <https://openlineage.io/docs/>
14. Great Expectations Core documentation: https://docs.greatexpectations.io/docs/core/introduction/try_gx/
15. Label Studio documentation: <https://labelstud.io/guide/>
16. Microsoft Presidio documentation: <https://microsoft.github.io/presidio/>
17. cleanlab documentation: <https://docs.cleanlab.ai/stable/>

18. Hugging Face Dataset Cards: https://huggingface.co/docs/datasets/dataset_card
19. MLflow Dataset Tracking: <https://mlflow.org/docs/latest/ml/dataset/>
20. MLCommons Croissant specification: <https://docs.mlcommons.org/croissant/docs/croissant-spec.html>
21. SPDX License List: <https://spdx.org/licenses/>
22. Schema.org Dataset type: <https://schema.org/Dataset>
23. DigitalOcean Spaces object storage: <https://www.digitalocean.com/products/object-storage>
24. Supabase Row Level Security docs: <https://supabase.com/docs/guides/database/postgres/row-level-security>
25. Next.js App Router docs: <https://nextjs.org/docs/app>

Final recommendation

The first implementation objective should be a full synthetic dataset dry-run inside the private admin platform: create a buyer brief, create a supplier asset, ingest a sample, profile it, run rights and PII gates, clean it, label a subset, generate QA, package a release, and record delivery evidence. That dry-run will expose the exact schema, UI, worker, and audit details needed before real client data is handled.